

Note: wherever a package is mentioned below, it is optional — all classes may instead be placed together in a single .java file.

EXERCISE 1 — ELECTRONIC DEVICE MANAGEMENT

1. Overview

Build a Java Console application to manage electronic devices sold in a store. Data is managed with an in-memory list (`ArrayList`). The program menu must run continuously until the user chooses to exit.

2. Class Structure

2.1. Abstract Class `ElectronicDevice` (parent class)

Manages the common information of every electronic device:

- Device ID
- Device Name
- Manufacturer
- Selling Price
- Import Date (`LocalDate importDate`, entered in yyyy-MM-dd format)

`ElectronicDevice` is an **abstract class**, declaring the abstract method `displayInfo()` which every child class must override (this must also display the import date).

Inside `ElectronicDevice` there is a **static nested class** `DeviceStatistics` responsible for statistics:

- Count of total devices
- Count of Phones
- Count of Laptops
- Average selling price of all devices

2.2. Class `Phone` (extends `ElectronicDevice`)

Additional attributes:

- Battery Capacity (must be greater than 0)
- 5G Support (yes/no)

Overrides `displayInfo()`. Implements the `Discountable` interface with a **10%** discount.

2.3. Class `Laptop` (extends `ElectronicDevice`)

Additional attributes:

- RAM (must be greater than 0)

- Screen Size

Overrides `displayInfo()`. Implements the `Discountable` interface with a **15%** discount.

2.4. Interface `Discountable`

```
double calculateDiscountPrice();
```

3. Custom Exceptions

Exception	When it is thrown
<code>DuplicateDeviceIdException</code>	The user enters a device ID that already exists when adding a new device
<code>InvalidPriceException</code>	The entered selling price is not greater than 0
Exception for invalid type-specific attribute	Battery capacity (Phone) or RAM (Laptop) is not greater than 0
<code>DeviceNotFoundException</code>	No device is found matching the requested ID

The program must use `try-catch` to handle incorrect input data types; it must not terminate abnormally when an error occurs, and must instead allow the user to re-enter the value.

4. Program Functions (consolidated menu)

CRUD

1. Add a device (Phone or Laptop) — check that the ID is not a duplicate (`DuplicateDeviceIdException`), price > 0 (`InvalidPriceException`), the type-specific attribute is valid, and enter the import date in yyyy-MM-dd format.
2. Search for a device by ID — returns `Optional<ElectronicDevice>` (using the Stream API); if found, display the information, otherwise display a “not found” message (no exception thrown on this normal search path).

Display / Filtering (using Lambda + Stream API)

3. Display the list of phones that support 5G.
4. Display the list of laptops with RAM of 16GB or more.
5. Find the phone with the largest battery capacity.
6. Find the laptop with the largest screen size.
7. Find the device with the highest selling price (using Stream + `Optional`; if the list is empty, show an appropriate message instead of an error).
8. Compute the total value of all devices.
9. Find devices by manufacturer (the user enters the manufacturer’s name).
10. Display the list of device names (using `map()`).
11. Display the list of devices imported in a year specified by the user.
12. Display the discounted price of all devices (ID, Name, Original Price, Discounted Price).

Sorting

13. Sort devices by name (A – Z) — implemented with a **Lambda Expression combined with a Comparator** when calling `Collections.sort()`/`List.sort()`. (No standalone *Comparator* class is created — this requirement supersedes the earlier approach of using a separate *Comparator* class.)
14. Sort devices by selling price in ascending order — implemented the same way, with a Lambda Expression.

Statistics

15. Display device statistics (via `DeviceStatistics`): total number of devices, number of Phones, number of Laptops, average selling price.

5. Consolidated Technical Requirements

- Apply inheritance to reuse common attributes; clearly separate the shared functionality from the type-specific functionality of each device type.
- `ElectronicDevice` is an abstract class; `Phone`/`Laptop` override `displayInfo()`.
- Apply the `Discountable` interface to `Phone` and `Laptop`.
- Apply Lambda Expressions and the Stream API to all filtering/searching/sorting/computation operations on the device list (replacing traditional for-loops and standalone *Comparator* classes).
- Use Method References instead of Lambda Expressions wherever appropriate.
- Use `Optional<ElectronicDevice>` for search functions.
- Use custom exceptions (`DuplicateDeviceIdException`, `InvalidPriceException`, `DeviceNotFoundException`, and the exception for invalid type-specific attributes) together with `try - catch` so the program never terminates abnormally.
- The `importDate` attribute is of type `LocalDate`, entered/displayed in yyyy-MM-dd format.

EXERCISE 2 — TRANSPORTATION MANAGEMENT SYSTEM

1. Overview

Build a Java Console application to manage transportation vehicles registered in a transportation center. All objects (both `Car` and `Motorcycle`) are stored in a single `ArrayList<Vehicle>`, and the program must run continuously until the user chooses to exit.

2. Class Structure

2.1. Abstract Class `Vehicle` (parent class)

Common information for every vehicle:

- Vehicle ID
- Vehicle Name
- Manufacturer

- Vehicle Value
- Registration Date (`LocalDate`, entered as `yyyy-MM-dd`, validated — invalid/incorrectly formatted input must be caught and re-prompted, e.g. *“Invalid date format. Please enter the date using yyyy-MM-dd.”*)

`Vehicle` is an **abstract class** implementing:

- `Comparable<Vehicle>` — natural ordering by Vehicle Value (ascending)
- `public abstract double calculateAnnualTax();` — overridden by each child class

Each child class must override a display method (e.g. `displayInfo()`) to show object-specific information, invoked polymorphically through a `Vehicle` reference wherever possible.

`Vehicle` also contains:

- A **static nested class** `VehicleStatistics`, automatically updated whenever a vehicle is added, exposing:
 - `public static int getTotalVehicles();`
 - `public static int getTotalCars();`
 - `public static int getTotalMotorcycles();`
 - `public static double getTotalVehicleValue();`
- A **non-static inner class** `VehicleInsurance` with attributes `Insurance Provider` and `Coverage Amount`. Every `Vehicle` object must contain exactly one `VehicleInsurance` object, entered by the user at creation time and shown as part of `displayInfo()`.

2.2. Class `Car` (extends `Vehicle`)

Additional attributes:

- Number of Seats (must be > 0)
- Fuel Type (enum: Gasoline / Diesel / Electric)

Annual tax (`calculateAnnualTax()`), based on fuel type:

Fuel Type	Tax Rate
Gasoline	5% of vehicle value
Diesel	6% of vehicle value
Electric	3% of vehicle value

Registration status (`getRegistrationStatus()`, from interface `Registrable`):

Condition	Status
Electric	Green Vehicle
Gasoline / Diesel	Standard Vehicle

2.3. Class Motorcycle (extends Vehicle)

Additional attributes:

- Engine Capacity in cc (must be > 0)
- ABS Supported (Yes/No)

Annual tax (calculateAnnualTax()), based on engine capacity:

Engine Capacity	Tax Rate
Less than 150cc	2% of vehicle value
150cc or greater	4% of vehicle value

Registration status (getRegistrationStatus()), from interface Registrable):

Condition	Status
ABS Supported	Safety Certified
No ABS	Basic Certified

2.4. Interface Registrable

```
public interface Registrable {  
    String getRegistrationStatus();  
}
```

Implemented by both Car and Motorcycle.

3. Custom Exceptions

Exception	Trigger
DuplicateVehicleIdException	User attempts to add a vehicle whose Vehicle ID already exists
InvalidVehicleValueException	Vehicle Value ≤ 0 , Number of Seats ≤ 0 , or Engine Capacity ≤ 0

All exceptions must be handled gracefully (appropriate error message, allow re-entry of the value); the program must keep running when an exception occurs, and must not crash on invalid-type user input.

4. Program Functions (consolidated menu)

CRUD / core operations

1. Add a Car — validated against DuplicateVehicleIdException and InvalidVehicleValueException; also collects Registration Date and VehicleInsurance info.
2. Add a Motorcycle — same validation and data collection as above.
3. Display All Vehicles.

4. Search Vehicle by ID — refactored to use Stream API and return `Optional<Vehicle>` (no more `null`); if found show full information, otherwise display “*Vehicle not found.*”
5. Search Vehicle by Name — same Stream API / `Optional<Vehicle>` approach.

Filtering (Stream API)

6. Display All Electric Cars.
7. Display All Motorcycles with ABS.
8. Display Vehicles by Manufacturer.

Sorting

9. Sort Vehicles by Value (ascending) — via natural ordering (`Comparable<Vehicle>`).
10. Sort by Vehicle Name (A → Z).
11. Sort by Manufacturer (A → Z).
12. Sort by Annual Tax (Highest → Lowest).

(Sorting is implemented directly with Lambda Expressions passed to `Collections.sort()`/`List.sort()`. The originally-specified standalone Comparator classes — `VehicleNameComparator`, `ManufacturerComparator`, `AnnualTaxComparator` — are superseded by this Lambda-based approach and must no longer be used.)

Reporting

13. Display Vehicle Tax Report — for each vehicle: Vehicle ID, Vehicle Name, Vehicle Type, Annual Tax, Registration Status (via polymorphic calls to `calculateAnnualTax()` and `getRegistrationStatus()`).
14. Display Vehicle Statistics — via `VehicleStatistics`: Total Vehicles, Total Cars, Total Motorcycles, Total Vehicle Value.
15. Display Vehicles Registered Within the Last N Days — user enters N; show vehicles whose registration date falls within that window from today.

5. Consolidated Technical Requirements

- The program must run continuously until the user chooses to exit; Vehicle IDs must be unique.
- Use inheritance to reuse common attributes/behavior; use polymorphism by storing both Car and Motorcycle in a single `ArrayList<Vehicle>`.
- `Vehicle` is abstract (`calculateAnnualTax()` abstract method) and implements `Comparable<Vehicle>`; Car/Motorcycle implement `Registrable`.
- Use the static nested class `VehicleStatistics` and the non-static inner class `VehicleInsurance` as specified above.
- All sorting is implemented with Lambda Expressions (no standalone Comparator classes); searching and filtering are implemented with the Stream API; method references replace lambdas where appropriate.

- Search operations return `Optional<Vehicle>` instead of `null`.
- Registration Date is a `LocalDate`, entered/validated as yyyy-MM-dd.
- All existing functionality from earlier stages of the assignment must remain unchanged and continue to work correctly as new features are layered on.

EXERCISE 3 — LIBRARY DOCUMENT MANAGEMENT (INTERFACE & ABSTRACT CLASS)

Overview

A library needs to manage documents including Books, Magazines, and Newspapers. Every document has the following common attributes: a unique Document ID, Publisher Name, and Number of Copies Issued.

- **Books** additionally track: Author Name, Number of Pages.
- **Magazines** additionally track: Issue Number, Issue Month.
- **Newspapers** additionally track: Issue Date.

Requirements

Requirement 1: Design the classes needed to manage the library's documents effectively, using an abstract class for the common attributes/behavior and an interface where appropriate.

Requirement 2: Build a `DocumentManager` class with the following functions:

- Add a new document: Book, Magazine, or Newspaper.
- Remove a document by Document ID.
- Display information about a document.
- Search for documents by type: Book, Magazine, or Newspaper.
- Exit the program.

EXERCISE 4 — UNIVERSITY ADMISSIONS (INTERFACE & ABSTRACT CLASS)

Overview

University entrance exam candidates sit for one of three exam groups: A, B, or C. Every candidate has: Candidate Number, Full Name, Address, and Priority Level.

- Group A candidates take: Math, Physics, Chemistry.
- Group B candidates take: Math, Chemistry, Biology.
- Group C candidates take: Literature, History, Geography.

Requirements

Requirement 1: Design the classes needed to manage exam candidates, using an abstract class for the shared candidate attributes and inheritance/interfaces for the group-specific behavior.

Requirement 2: Build an `AdmissionManager` class with the following functions:

- Add a new candidate.
- Display a candidate's information along with their exam group.
- Search for a candidate by Candidate Number.
- Exit the program.

EXERCISE 5 — DOCUMENT MANAGEMENT (INTERFACE)

The IDocument Interface

Create an interface named `IDocument`. The `IDocument` interface should declare the following methods: - `void input();` - `void display();`

The Book Class

Create a class named `Book`. The `Book` class must implement the `IDocument` interface and includes the following attributes: - `int id;` - `String bookTitle;` - `String authorName;` - `float price;`

Requirements: - All attributes must be declared `private`. - Create two constructors for the `Book` class: a no-argument constructor and a constructor with all 4 parameters to set the attribute values. - Create getter and setter methods for each attribute.

Implementing input() and display()

Implement the two `Book` methods inherited from the `IDocument` interface: - `input()`: prompts the user to enter `id`, `bookTitle`, `authorName`, and `price` for a `Book` object, then assigns the entered values to the corresponding attributes. - `display()`: displays the information of a `Book` object.

The DocumentManager Class

Create a class named `DocumentManager`. The `DocumentManager` class has the following methods: - `void addDocument()`: asks the user to enter the number of books `n` to store, then allows the user to enter information for `n` books into a `Book` array. - `void displayAllDocuments()`: displays all information of every `Book` object in the array. - `void searchByAuthor(String authorName)`: allows the user to enter an author's name, then searches the `Book` array and displays information of any matching books.

Testing and Menu System

Create a class named `DocumentTest` containing a `main` method, which performs the following tasks: - Displays a menu allowing the user to enter a choice:

1. Add new book
 2. Display all books
 3. Search books by author
 4. Exit
- If the user selects 1, call the `addDocument` method of the `DocumentManager` class. Return to the menu after completion.
 - If the user selects 2, call the `displayAllDocuments` method of the `DocumentManager` class. Return to the menu after completion.
 - If the user selects 3, prompt the user to enter an author's name, then call `searchByAuthor` with the entered name. Return to the menu after completion.
 - If the user selects 4, end the program.

EXERCISE 6 — OUTDOOR TRIP MANAGEMENT SYSTEM

Overview

You will build a simple outdoor trip management system by applying object-oriented programming (OOP) principles. The system manages trips with attributes such as trip ID, destination, departure date, and number of participants.

Abstract Class for Trip

a) Create the abstract class `Trip`

Create an abstract class named `Trip`. This class should have the following attributes:

- `String id` (protected)
- `String destination` (protected)
- `String date` (protected)
- `String numberOfParticipants` (protected)

Create two constructors: a default constructor and a parameterized constructor.

Create getter and setter methods for the attributes. Define two abstract methods: - `public abstract void enterInfo();` - `public abstract void showInfo();`

b) Create the `FamilyTrip` class

Create a class named `FamilyTrip`, inheriting from `Trip`. Add the following attributes:

- `boolean hasChildren` (private)
- `boolean familyType` (private)

Create two constructors: a no-argument constructor and a 6-argument constructor to set values for both inherited and new attributes, using `super` to call the parent constructor.

Create getters and setters for the new attributes. Implement `enterInfo()` and `showInfo()` inherited from `Trip`: - `enterInfo()`: prompts the user to enter values for all attributes of the `FamilyTrip` object. - `showInfo()`: prints all attributes of the `FamilyTrip` object.

AdventureTrip Class

a) Create the AdventureTrip class

Create a class named `AdventureTrip`, inheriting from `Trip`. Add the following attributes:

- `String activityType` (private)
- `String difficultyLevel` (private)

Create two constructors: a no-argument constructor and a 6-argument constructor to set values for inherited and new attributes, using `super` to call the parent constructor.

Create getters and setters for the new attributes. Implement `enterInfo()` and `showInfo()` inherited from `Trip`: - `enterInfo()`: prompts the user to enter values for all attributes of the `AdventureTrip` object. - `showInfo()`: prints all attributes of the `AdventureTrip` object.

b) Calculate Estimated Trip Duration

Create a new method `estimateDuration()` in the `AdventureTrip` class that calculates the estimated duration of the trip based on the activity type and difficulty level:

- Hiking: 1 day per difficulty level.
- Rafting: 1.5 days per difficulty level.
- Mountain Climbing: 2 days per difficulty level.

Example: if the activity is “Hiking” with difficulty 4, the estimated duration is 4 days ($1 \text{ day} \times 4$). If the activity is “Rafting” with difficulty 3, the estimated duration is 4.5 days ($1.5 \text{ days} \times 3$).

Implement `estimateDuration()` to return the estimated number of days based on the criteria above. After calling `showInfo()`, also print the estimated trip duration using this method.

Expected result:

```
Trip ID: TR001
Destination: Grand Canyon
Activity Type: Hiking
Difficulty Level: 4
Estimated Trip Duration: 4 days
```

Testing and Menu System

a) Main menu

Create a class named `TripTest`, containing a `main()` method that implements a menu-driven program:

Please select:

1. Enter information for n family trips.
2. Enter information for n adventure trips.
3. Display information for n family trips (sorted by number of participants, descending).
4. Display information for n adventure trips (sorted by difficulty level, ascending).
5. Exit.

Your choice:

b) Input family trips

When the user selects option 1, prompt the user to enter the number of family trips and store their information in an array of `FamilyTrip` objects.

c) Input adventure trips

When the user selects option 2, prompt the user to enter the number of adventure trips and store their information in an array of `AdventureTrip` objects.

d) Display sorted family trips

When the user selects option 3, display the list of `FamilyTrip` objects sorted by number of participants in descending order.

e) Display sorted adventure trips

When the user selects option 4, display the list of `AdventureTrip` objects sorted by difficulty level in ascending order.

f) Exit the program

When the user selects option 5, exit the program.

EXERCISE 7 — HOTEL MANAGEMENT (ABSTRACT CLASS & INTERFACE)

Overview

To manage guests renting rooms at a hotel, the following information is needed: number of nights rented, room type, and personal information of each guest. Room type A costs \$500, room type B costs \$300, and room type C costs \$100. For each individual, the following information must be managed: full name, age, and ID number.

Requirements

Requirement 1:

Build an abstract class `PersonAbs` with attributes `name` and `age`, and one abstract method `goToWork()`. Build a class `Person` to manage the personal information of the guests, inheriting from `PersonAbs` and adding an `ID` number attribute. In `goToWork()`, print the line “go to work”.

Requirement 2:

Build an interface `IHotel` with 3 methods: `addCustomer`, `deleteCustomer`, `calculatePrice`, representing the hotel's operations.

Build a `Hotel` class implementing `IHotel` to manage the information about the rooms in the hotel.

Requirement 3:

Build methods to add a new guest and remove a guest by `ID` number. Calculate the room charge for a guest (identified by `ID` number) using the formula: `number of nights rented × price of the room type`.

EXERCISE 8 — TRANSPORTATION MANAGEMENT (INTERFACE)

Overview

The traffic police department needs to manage transportation vehicles including: cars, motorcycles, and trucks. Build a program to manage vehicles as follows: create an interface `IVehicle` defining 3 methods:

```
void startEngine();  
void stopEngine();  
void brake();
```

A `Vehicle` class implements `IVehicle`. Three classes — `Car`, `Motorcycle`, and `Truck` — inherit from `Vehicle`. Each type includes the following information: `ID`, `Manufacturer`, `Year of Manufacture`, `Selling Price`, and `Color`.

Requirements

Cars have their own attribute: `Number of Seats (Integer)`, `Engine Type (String)`.

Motorcycles have their own attribute: `Engine Power (Integer)`.

Trucks additionally manage: `Load Capacity (Integer)`.

For the three classes (`Car`, `Motorcycle`, `Truck`), implement type-specific methods as follows: when the `Motorcycle`'s `startEngine()` method runs, it prints “Starting motorcycle engine”. The other two classes behave similarly for their own `startEngine()` methods.

The Truck class additionally implements the method:

```
Integer loadCargo(Integer weight);
```

In this method, if the given weight is greater than the truck's load capacity, print "Load weight is not suitable"; otherwise, print "Load weight is suitable".

The Car class additionally implements the method:

```
Integer boardPassengers(Integer numberOfPassengers);
```

This method checks whether the number of passengers exceeds the number of seats in the car, prints the result, and returns the number of passengers that exceed capacity at the end of the method.

- **Requirement 1:** Build the classes needed to manage the above vehicles efficiently.
- **Requirement 2:** Build a `VehicleManager` class with the following functions:
 - Add/remove (by ID) vehicles of the above types.
 - Call the `loadCargo` function of a truck by ID.
 - Call the `boardPassengers` function of a car by ID.
 - Search for vehicles by manufacturer or color.
 - Exit the program.

EXERCISE 9 — WAREHOUSE MANAGEMENT (INTERFACE & INHERITANCE)

Overview

Build a warehouse management system that handles different product categories such as food, electronics, and household goods.

1. Create the Interface and Classes

Interface `IProduct`: defines the following methods:

```
void addStock(int quantity);  
void removeStock(int quantity);  
void displayInfo();
```

Class `Product`: implements `IProduct`. Includes the attributes:

```
private String id;  
private String name;  
private double price;  
private int quantity;
```

Concrete classes inheriting from `Product`:

- **Food:** adds the attributes:

```
private String expirationDate;  
private boolean isPerishable;
```

Provides the method:

```
public void updateExpirationDate(String expirationDate);  
public void displayInfo();
```

- **Electronics:** adds the attributes:

```
private String warrantyPeriod;  
private String brand;
```

Provides the method:

```
public void extendWarranty(String additionalPeriod);  
public void displayInfo();
```

- **Household:** adds the attributes:

```
private String material;  
private String usage;
```

Provides the method:

```
public void updateUsage(String usage);  
public void displayInfo();
```

2. Create the InventoryManager Class

Manages products with the following functions:

- **Add, remove** (by ID).
- **Find products** by name and category (food, electronics, household).
- **Call updateExpirationDate** on a Food item by ID.
- **Call extendWarranty** on an Electronics item by ID.
- **Update usage information** of a Household item by ID.
- **Display information** for all products.

3. Additional Requirements

- Each product type must have a different implementation of `displayInfo()`. For example, displaying a food item shows expiration and perishability information, and similarly for the other types.
- Update product-specific information (expiration date, warranty period, or usage) through dedicated methods.
- Create a `Main` class with an interactive menu allowing the user to choose functions such as add, remove, search, and display product information.
- Use an appropriate data structure such as a `HashMap` to store products and process requests efficiently.

EXERCISE 10 — STUDENT MANAGEMENT (INTERFACE)

The IStudent Interface

Create an interface named `IStudent`, with the following methods:

Return Type	Name
<code>void</code>	<code>input</code>
<code>void</code>	<code>display</code>

The Student Class

Create a class named `Student`, implementing the `IStudent` interface, with the following attributes:

Data Type	Name
<code>int</code>	<code>id</code>
<code>String</code>	<code>fullname</code>
<code>String</code>	<code>email</code>
<code>float</code>	<code>mark</code>

- Create 2 constructors for the `Student` class: a no-argument constructor, and a constructor with all 4 parameters to set the attribute values.
- Create getter/setter methods for each attribute.
- Implement the `input()` and `display()` methods inherited from the `IStudent` interface:
 - `input()`: prompts the user to enter values for all attributes of the `Student` object.
 - `display()`: prints all attributes of the `Student` object.

The StudentManagement Class

Create a class `StudentManagement`, with the following methods:

Modifier	Return Type	Name
<code>public</code>	<code>void</code>	<code>addStudent</code>
<code>public</code>	<code>void</code>	<code>showStudent</code>
<code>public</code>	<code>Student</code>	<code>sortStudentByMark</code>

- `addStudent`: prompts the user to enter 3 `Student` objects into a `Student` array.
- `showStudent`: iterates over every element of the `Student` array and displays each object.
- `sortStudentByMark`: sorts the `Student` array by mark.

The MainClass and Menu System

Create a class `MainClass`, with a `main` method to run the application. The `main` method performs the following tasks:

Display a menu for the user to choose from:

```
=== MENU ===
```

1. Add New.
2. Show All.
3. Sort.
4. Exit.

Your choice:

- When the user selects 1: run the `addStudent` method of `StudentManagement`.
- When the user selects 2: run the `showStudent` method of `StudentManagement`.
- When the user selects 3: run the `sortStudentByMark` method of `StudentManagement`.
- When the user selects 4: exit the program.

EXERCISE 11 — BOOK LIBRARY MANAGEMENT (INTERFACE & INHERITANCE)

Overview

Build a book library management system that handles different book categories such as textbooks, novels, and reference books.

1. Create the Interface and Classes

Interface `IBook`: defines the following methods:

```
void addCopies(int quantity);  
void removeCopies(int quantity);  
void displayInfo();
```

Class `Book`: implements `IBook`. Includes the attributes:

```
private String id;  
private String title;  
private String author;  
private double price;  
private int quantity;
```

Concrete classes inheriting from `Book`:

- **Textbook:** adds the attributes:

```
private String subject;  
private String academicLevel;
```

Provides the method:

```
public void updateAcademicLevel(String academicLevel);  
public void displayInfo();
```

- **Novel**: adds the attributes:

```
private String genre;  
private boolean isBestseller;
```

Provides the method:

```
public void markAsBestseller();  
public void displayInfo();
```

- **ReferenceBook**: adds the attributes:

```
private String fieldOfStudy;  
private String edition;
```

Provides the method:

```
public void updateEdition(String edition);  
public void displayInfo();
```

2. Create the LibraryManager Class

Manages the books with the following functions:

- **Add, remove** (by ID).
- **Find books** by title and category (textbook, novel, reference).
- **Update the academic level** of a Textbook by ID.
- **Mark a Novel as bestseller** by ID.
- **Update the edition** of a ReferenceBook by ID.
- **Display information** for all books.

EXERCISE 12 — HOSPITAL DOCTOR MANAGEMENT (ABSTRACT CLASS)

Abstract Class for Doctor

a) Create the abstract class Doctor

Create an abstract class named Doctor. This class should contain the following attributes:

Data Type	Name	Access Modifier
String	doctorId	protected

Data Type	Name	Access Modifier
String	doctorName	protected
String	specialization	protected
float	salary	protected

Create two constructors: a default constructor and a parameterized constructor. Create getter and setter methods for the attributes. Define two abstract methods: - public abstract void input(); - public abstract void display();

b) Create the SpecialistDoctor class

Create a class named SpecialistDoctor, inheriting from Doctor. Add the following attributes:

Data Type	Name	Access Modifier
String	expertise	private
int	yearsOfExperience	private

Create two constructors: a no-argument constructor and a 6-argument constructor to set values for both inherited and new attributes, using this to call the parent constructor. Create getters and setters for the new attributes. Implement input() and display() inherited from Doctor: - input(): prompts the user to enter values for all attributes of the SpecialistDoctor object. - display(): prints all attributes of the SpecialistDoctor object.

Surgeon Class

a) Create the Surgeon class

Create a class named Surgeon, inheriting from Doctor. Add the following attributes:

Data Type	Name	Access Modifier
int	numberOfSurgeries	private
String	hospitalName	private

Create two constructors: a no-argument constructor and a full-argument constructor to set values for both inherited and new attributes, using this to call the parent constructor. Create getters and setters for the new attributes. Implement input() and display() inherited from Doctor: - input(): prompts the user to enter values for all attributes of the Surgeon object. - display(): prints all attributes of the Surgeon object.

Testing and Menu System

a) Main menu

Create a class DoctorTest, containing a main method that implements a menu-driven program:

Please select:

1. Enter information for n specialist doctors.
2. Enter information for n surgeons.
3. Display information for n specialist doctors (sorted by salary, descending).
4. Display information for n surgeons (sorted by number of surgeries, ascending).
5. Exit.

Your choice:

b) Input specialist doctors

When the user selects option 1, prompt the user to enter the number of specialist doctors and store their information in an array of `SpecialistDoctor` objects.

c) Input surgeons

When the user selects option 2, prompt the user to enter the number of surgeons and store their information in an array of `Surgeon` objects.

d) Display sorted specialist doctors

When the user selects option 3, display the list of `SpecialistDoctor` objects sorted by salary in descending order.

e) Display sorted surgeons

When the user selects option 4, display the list of `Surgeon` objects sorted by number of surgeries performed in ascending order.

f) Exit the program

When the user selects option 5, exit the program.

EXERCISE 13 — COMPUTER & BOOK PRODUCT CATALOG (ABSTRACT CLASS)

The Abstract Product Class

Create an abstract class named `Product`, with the following fields:

#	Data Type	Name	Access Modifier
1	String	proId	protected
2	String	proName	protected
3	int	year	protected
4	float	price	protected

Create 2 constructors, and getters/setters for the attributes. Create 2 abstract methods: - `public abstract void input();` - `public abstract void display();`

The Computer Class

Create a class named `Computer`, inheriting from `Product`, with the attributes:

#	Data Type	Name	Access Modifier
1	String	speed	private
2	String	producer	private

Create 2 constructors for this class: - No-argument. - 6-argument, to initialize both the inherited and the added attributes. You must use the `this` keyword to call the parent's parameterized constructor.

Create getters/setters for the added attributes. Implement the `input()` and `display()` methods inherited from `Product`: - `input()`: allows the user to enter values for both the inherited and added attributes of `Computer`. - `display()`: prints all attributes of the `Computer` object.

The Book Class

Create a class named `Book`, inheriting from `Product`, with the attributes:

#	Data Type	Name	Access Modifier
1	String	type	private
2	String	publisher	private

Create 2 constructors for this class: - No-argument. - 6-argument, to initialize all attributes for this object. You must use the `this` keyword to call the constructor from the parent class.

Create getter/setter methods for the attributes. Implement the `input()` and `display()` methods inherited from `Product`: - `input()`: allows the user to enter values for all inherited and added attributes of the `Book` object. - `display()`: prints all inherited and added attributes of the `Book` object.

Testing and Menu System

Create a class `Test`, with a main method to run the application. The main method should perform the following tasks:

1. Display a menu for the user to select from:
Please select:
 1. Input information for n Computers.
 2. Input information for n Books.
 3. Display information of n Computers by sorting the price descending.
 4. Display information of n Books by sorting the publisher ascending.
 5. Exit.
- Your choice:

2. When the user selects 1: Enter information for n computers into an array of type `Computer`.
3. When the user selects 2: Enter information for n books into an array of type `Book`.
4. When the user selects 3: Display the information in the `Computer` array above, sorted.
5. When the user selects 4: Display the information in the `Book` array above, sorted.
6. When the user selects 5: Exit the program.

EXERCISE 14 — EXCEPTION HANDLING PRACTICE

Write small Java programs that use `try-catch-finally` blocks to handle each of the following exception scenarios:

1. Handle the exception raised when the user enters data of the wrong type.
2. Handle the exception raised when dividing a number by zero.
3. Handle the exception raised when trying to open a file that does not exist.
4. Handle the exception raised when accessing an index that does not exist in an array.
5. Handle the exception raised when converting a non-numeric string into a numeric type.
6. Handle the exception raised when a method cannot find a file.
7. Write a program that uses `try-catch-finally` together with a `throws` declaration to handle the exception when dividing a number by zero.
8. Write a program that uses the `assert` keyword to check whether an input value is a positive number.